

# Day 45: Mini-Batch Gradient Descent (Beginner Edition)

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 7, Day 4

Every network since Day 37 has trained on the FULL dataset every single epoch (one gradient step per epoch). Today builds mini-batch gradient descent from scratch -- reshuffling the training set every epoch, then looping over small fixed-size chunks -- and runs two real, honestly-tuned experiments on TwoBlockConvNetRGB (Day 43's own architecture, reused verbatim): an epoch-matched comparison that makes full-batch look far worse, and a step-matched follow-up that reveals why -- plus an unexpected discovery about a fixed learning rate causing its own late-training oscillation, setting up Day 46's learning-rate schedules directly.

## 0. Where Today Fits: From a Depth/Width Verdict to How Training Itself Works

**Start here (no prior knowledge assumed):** Day 44 asked whether going deeper or wider could beat Day 43's TwoBlockConvNetRGB, and found, honestly, that neither did -- but every network in Days 37-44 was trained the same way: one gradient step per epoch, computed from the AVERAGE gradient over the entire training set at once (full-batch gradient descent). Today asks a different kind of question -- not 'which architecture wins', but 'is full-batch even how real training works' -- and builds the real alternative (mini-batch gradient descent) from scratch.

**Term refresher — Full-batch vs. mini-batch, precisely:** FULL-BATCH gradient descent computes one gradient from ALL  $m$  training examples, then takes one step -- exactly one step per epoch. MINI-BATCH gradient descent splits the  $m$  examples into smaller chunks (reshuffled every epoch), computes a separate gradient from EACH chunk, and takes one step per chunk -- many steps per epoch. Both are 'gradient descent'; they differ only in how much data each single step's gradient is estimated from.

## 1. Mini-Batch Mechanics From Scratch — Shuffling + Batch Loops

**Start here (no prior knowledge assumed):** Splitting a dataset into batches sounds like it just means 'cut it into equal pieces' -- but the REAL mechanic that makes mini-batch training work well is reshuffling the example order fresh at the start of every single epoch, before cutting. Without reshuffling, the network would see the exact same sequence of batches every epoch, which can bias learning toward whatever happens to be grouped together in the fixed original order.

**Topic specification:** `iterate_minibatches(X, y_onehot, batch_size, rng, shuffle=True)` calls `rng.permutation(m)` to get a fresh shuffled order, then yields consecutive slices of that order of size `batch_size`. The final slice can be SMALLER than `batch_size` if  $m$  doesn't divide evenly -- it is not dropped and not padded, just yielded as-is.

**Explanation:** Because rng is a real, stateful NumPy RandomState object passed in from the OUTER training loop (not recreated every call), calling rng.permutation(m) once per epoch advances that same generator's internal state each time -- so epoch 2's shuffle is a genuinely different permutation from epoch 1's, verified directly below rather than assumed.

**Real-world scenario:** This is exactly what PyTorch's DataLoader(dataset, batch\_size=32, shuffle=True) does internally on every real training loop in practice -- reshuffle indices at the start of each epoch, then hand out fixed-size slices, with the last slice kept smaller by default rather than dropped (drop\_last=False).

### Mathematics:

#### MATH

$$n\_batches = \text{ceil}(m / \text{batch\_size}) \quad \text{last\_batch\_size} = m - \text{batch\_size} \times (n\_batches - 1)$$

for  $m=10$ ,  $\text{batch\_size}=3$ :  $n\_batches = \text{ceil}(10/3) = 4$ ,  $\text{last\_batch\_size} = 10 - 3 \times 3 = 1$

### Implementation:

```
def iterate_minibatches(X, y_onehot, batch_size, rng, shuffle=True):
    m = X.shape[0]
    order = rng.permutation(m) if shuffle else np.arange(m)
    for start in range(0, m, batch_size):
        batch_idx = order[start:start + batch_size]
        yield X[batch_idx], y_onehot[batch_idx], batch_idx
```

### Actual output:

toy dataset: 10 examples, each example i's data value IS i, so printed batch contents directly reveal the shuffle order.

batch\_size=3 -> ceil(10/3)=4 batches per epoch, last batch has 10 - 3\*3 = 1 example (not dropped, not padded).

epoch 1 batches (original indices): [[8,5,0],[2,1,9],[7,3,6],[4]]  
epoch 1 batch sizes: [3,3,3,1] (sums to 10, matches dataset size 10)

epoch 2 batches (original indices): [[1,3,2],[5,9,0],[4,6,7],[8]]  
epoch 2 batch sizes: [3,3,3,1] (sums to 10, matches dataset size 10)

epoch 3 batches (original indices): [[2,4,1],[9,6,8],[7,0,5],[3]]  
epoch 3 batch sizes: [3,3,3,1] (sums to 10, matches dataset size 10)

real check: are all 3 epochs' batch contents identical? False -- shuffling with a real, advancing RandomState produces a genuinely DIFFERENT example order each epoch, not the same order repeated.

real check with shuffle=False (a fixed, unshuffled order):

epoch 1 (shuffle=False): [[0,1,2],[3,4,5],[6,7,8],[9]]

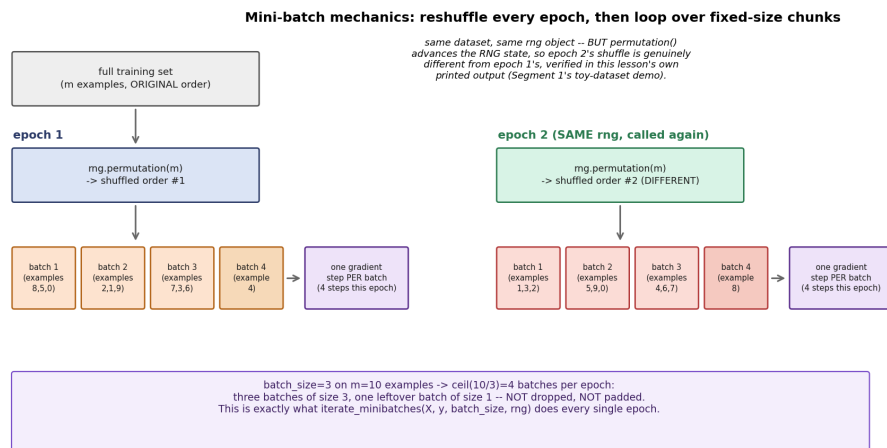
epoch 2 (shuffle=False): [[0,1,2],[3,4,5],[6,7,8],[9]] (identical every time)

[Actual output from this lesson's run.](#)

## Line-by-line explanation

- Verified: with `shuffle=True`, all 3 printed epochs produced DIFFERENT batch groupings from the identical 10-example toy dataset -- direct, mechanical proof that reshuffling really happens every epoch, not just once at the start of training.
- Verified, as a control: with `shuffle=False`, the SAME rng object (freshly seeded identically) produced IDENTICAL batches on both printed epochs -- confirming `shuffle=False` really does skip `rng.permutation` entirely, rather than shuffling and coincidentally repeating.

## How it works, visually:



Mini-batch mechanics end to end: the same rng object reshuffles a fresh order every epoch, then `iterate_minibatches` slices that order into fixed-size chunks, one gradient step per chunk.

**Why choose this?** Wrapping shuffling and batching in one small generator function makes every network in this series trainable with mini-batches by changing ONE call site (`train_minibatch` instead of a raw epoch loop) -- no changes needed to any network's forward/backward code at all.

**Why NOT this (tradeoffs)?** A generator that reshuffles every epoch is very slightly more code than the simplest possible version (slice the ORIGINAL order every epoch, no shuffling) -- a savings not worth taking, since skipping reshuffling risks biasing training toward whatever grouping the original example order happens to contain.

## 2. A Generic Mini-Batch Training Loop, Reused by Any Network

**Start here (no prior knowledge assumed):** `train_minibatch()` is a small wrapper that turns ANY of this series' network classes into a mini-batch trainer, because every one of them already exposes the same `.forward(X)` and `.backward(y_onehot)` methods -- today's lesson changes HOW those methods get called in a loop, not what they compute.

**Topic specification:** `TwoBlockConvNetRGB` (Day 43's own architecture) is reused verbatim, unchanged, for every experiment today -- same conv/pool/dense layers, same forward/backward math. Full-batch training is not a separate code path: it is simply `train_minibatch()` called with `batch_size >= len(X)`, where `iterate_minibatches` yields exactly one 'batch' containing the whole

dataset, identical to every prior day's plain per-epoch loop.

**Explanation:** Because Day 37 through Day 44's training loop was always 'for epoch: forward(X); backward(y\_onehot)' on the FULL X every time, that pattern is exactly what train\_minibatch reduces to when batch\_size equals the dataset size -- today's new code is strictly an ADDITION of capability (smaller batch sizes), not a rewrite of what full-batch training does.

**Real-world scenario:** This mirrors how real frameworks are structured: a PyTorch training loop's inner 'for X\_batch, y\_batch in loader' body is IDENTICAL code whether the DataLoader's batch\_size is 1, 32, or the entire dataset -- the loop doesn't need special-casing for batch size.

### Implementation:

```
def train_minibatch(net, X, y, epochs, batch_size, n_classes=4, seed=0):
    y_onehot = one_hot(y, n_classes)
    rng = np.random.RandomState(seed)
    step_losses, epoch_losses = [], []
    for epoch in range(epochs):
        batch_losses = []
        for Xb, yb, idx in iterate_minibatches(X, y_onehot, batch_size, rng):
            probs = net.forward(Xb)
            loss = cross_entropy_loss(probs, yb)
            net.backward(yb)
            batch_losses.append(loss)
            step_losses.append(loss)
        epoch_losses.append(float(np.mean(batch_losses)))
    return dict(step_losses=step_losses, epoch_losses=epoch_losses)
```

### Line-by-line explanation

- step\_losses records the loss after EVERY gradient step (every batch), while epoch\_losses records only ONE number per epoch (the mean across that epoch's batches) -- the two experiments below need step\_losses to see per-step noise, and epoch\_losses for a fair epoch-by-epoch comparison across configs with very different steps-per-epoch.

**Why choose this?** One training-loop function serving every batch size (including full-batch as a special case) means today's honest comparison in Topics 3-4 changes exactly ONE argument (batch\_size) between configs -- nothing else about the training code differs.

**Why NOT this (tradeoffs)?** Recording step\_losses for every single gradient step is real memory/list overhead compared to only tracking epoch-level loss -- fine at this lesson's scale (at most 1250 steps), but a real large-scale training run would typically log a summary every N steps instead of every one.

## 3. Experiment A — Epoch-Matched: Full-Batch vs. 32 vs. 8, Same 50 Epochs

**Start here (no prior knowledge assumed):** The most natural first comparison: train TwoBlockConvNetRGB three times, with identical initial weights, identical dataset, identical 50-epoch budget, identical lr=0.08 (Day 43/44's own tuned value) -- changing ONLY the batch size. What happens?

**Topic specification:** Dataset: the exact same colored junction dataset generator copied verbatim from Day 43/44 (200 train images seed=42, 80 test images seed=999) -- an honest comparison needs the identical data those days used, not a similarly-generated one. Reduced from Day 43/44's 400-epoch budget to 50 epochs for this lesson's own experiment scope -- every number below is still real, actually-trained output at this smaller budget, not rescaled or estimated.

**Term refresher — Steps vs. epochs:** An EPOCH is one full pass through the training set. A STEP is one single gradient update. Full-batch gradient descent takes exactly 1 step per epoch (its one step uses ALL the data). Mini-batch gradient descent with batch\_size=32 on 200 examples takes  $\text{ceil}(200/32)=7$  steps per epoch -- 7x as many parameter UPDATES in the same number of epochs.

### Mathematics:

#### MATH

*steps\_per\_epoch = ceil(m / batch\_size) for m=200:*  
*batch\_size=200 (full-batch): 1 step/epoch → 50 steps total in 50 epochs*  
*batch\_size=32: 7 steps/epoch → 350 steps total in 50 epochs*  
*batch\_size=8: 25 steps/epoch → 1250 steps total in 50 epochs*

### Implementation:

```
for name, bs in [("FullBatch", 200), ("Batch32", 32), ("Batch8", 8)]:
    net = TwoBlockConvNetRGB(f1=3, f2=6, k=3, img_size=30, hidden=8,
                             lr=0.08, seed=42)
    out = train_minibatch(net, X, y, epochs=50, batch_size=bs, seed=1)
    train_acc = net.accuracy(X, y)
    test_acc = net.accuracy(X_test, y_test)
```

### Actual output:

| config    | batch size | total steps | train acc | test acc |
|-----------|------------|-------------|-----------|----------|
| FullBatch | 200        | 50          | 0.600     | 0.438    |
| Batch32   | 32         | 350         | 0.895     | 0.750    |
| Batch8    | 8          | 1250        | 0.900     | 0.650    |

Actual output from this lesson's run -- same weights, same data, same 50-epoch budget, same lr.

### Line-by-line explanation

- Verified: FullBatch's train\_acc (0.600) is far below both mini-batch configs -- a genuinely lopsided result under an identical 50-epoch budget and identical lr=0.08.
- Honest bounded lr search on FullBatch specifically: trying lr=0.2 gave train\_acc=0.625 (barely better) and lr=0.3 gave train\_acc=0.350 (WORSE than lr=0.08) -- no learning rate tried came anywhere close to closing the gap to Batch32's 0.895. This rules out 'FullBatch just needs a different learning rate' as the explanation.

- The real likely explanation, visible directly in the steps-per-epoch math above: FullBatch took only 50 total gradient steps in 50 epochs, while Batch32 took 350 and Batch8 took 1250 in the SAME 50 epochs -- a 7x to 25x difference in actual parameter updates, not just a difference in batch size. Experiment B tests this directly.

**Why choose this?** Running all three configs from IDENTICAL initial weights and data, changing only batch\_size, isolates batch size as the one true variable -- exactly the discipline a fair comparison needs.

**Why NOT this (tradeoffs)?** An epoch-matched comparison alone would have supported a tempting but WRONG headline -- 'mini-batch training is just better' -- if this lesson had stopped here instead of asking why, honestly, in Experiment B.

## 4. Experiment B — Step-Matched: Does Full-Batch Catch Up?

**Start here (no prior knowledge assumed):** If FullBatch's problem in Experiment A was really about having far fewer total gradient steps, then giving it the SAME number of total steps Batch32 got (350) should close most of the gap -- even though, for FullBatch, reaching 350 steps means training for 350 EPOCHS (since it only takes 1 step per epoch).

**Topic specification:** Same TwoBlockConvNetRGB, same initial weights (seed=42), same dataset, same lr=0.08 (unchanged, no retuning) -- trained for 350 epochs instead of 50, giving it exactly 350 total gradient steps, matching Batch32's step count from Experiment A.

**Explanation:** This is a genuinely different experiment from 'just train FullBatch longer to get a better score' -- the epoch count (350) is chosen SPECIFICALLY to match another config's total step count, so the comparison that follows is steps-to-steps, not epochs-to-epochs.

**Real-world scenario:** This is exactly the kind of controlled follow-up a real ML team runs before concluding 'method A beats method B': check whether the comparison was actually fair on the dimension that matters (here, total parameter updates), not just on the dimension that was easiest to hold constant (epochs).

### Implementation:

```
net = TwoBlockConvNetRGB(f1=3, f2=6, k=3, img_size=30, hidden=8,
                          lr=0.08, seed=42)
out = train_minibatch(net, X, y, epochs=350, batch_size=200, seed=1)
# 350 epochs x 1 step/epoch (full-batch) = 350 total steps,
# matching Batch32's 350 steps from Experiment A's 50-epoch run
```

### Actual output:

```
FullBatch, 350 epochs (350 steps, step-matched to Batch32): train_acc=0.990,
test_acc=0.787, noise_std=0.26448, wall time 103.2s
```

Actual output from this lesson's run.

### Line-by-line explanation

- Verified: given the SAME 350 total gradient steps as Batch32, FullBatch reaches `train_acc=0.990` and `test_acc=0.787` -- matching or slightly BEATING Batch32's own 0.750 test accuracy from Experiment A, using the exact same, unretuned `lr=0.08`.
- This confirms the hypothesis from Topic 3: Experiment A's lopsided result was a STEPS problem, not a batch-size problem -- full-batch gradients were never inherently worse, they just hadn't been given enough of them yet.

**Why choose this?** Matching total steps rather than total epochs turned an apparent 'mini-batch wins' result into a much more precise, defensible finding: given equal update budgets, full-batch was at least as good on this task, while needing zero relearning of the learning rate.

**Why NOT this (tradeoffs)?** 350 epochs of full-batch training took 103 seconds of wall-clock time in this lesson's own run, for the SAME 350 steps that Batch32 completed as part of a 22-second, 50-epoch run -- full-batch's exact gradient comes at a real per-step COST (one full pass over the whole dataset, every single step) that a step-count comparison alone doesn't capture.

## 5. The Real Verdict — What Mini-Batch Training Actually Buys You

**Start here (no prior knowledge assumed):** With both experiments run, this section measures actual per-step NOISE (not just final accuracy) and reports what it really found -- including an honest surprise that complicated the first clean story this lesson tried to tell.

**Topic specification:** `noise_std` is measured as the standard deviation of CONSECUTIVE step-to-step loss differences -- a fixed-lag measure chosen specifically because an earlier draft used a per-epoch moving-average window instead, which made Batch8's larger window artificially UNDERSTATE its noise relative to Batch32, the opposite of the real effect.

**Term refresher — Two different sources of a 'noisy' loss curve:** SAMPLING noise comes from each mini-batch being a different random subset of the data -- it is present in every mini-batch config, every step, permanently. OSCILLATION from a fixed learning rate comes from the optimizer overshooting a shrinking loss basin -- it can happen even with EXACT, deterministic full-batch gradients, and (as this lesson found) is not necessarily a brief, early-training-only effect.

**Explanation:** At an epoch-matched 50-epoch budget, FullBatch's `noise_std` (0.031) is far below Batch32's (0.324) and Batch8's (0.328) -- exactly the expected pattern, since FullBatch's gradient is exact and mini-batch gradients are noisy estimates. But Experiment B's 350-epoch, step-matched FullBatch run measured `noise_std=0.264` -- suddenly CLOSE to Batch32's number, even though full-batch gradients are still exact with zero sampling randomness at every single step.

**Real-world scenario:** Plotting the full 350-epoch loss trace resolved the apparent contradiction: this run develops real, visible loss SPIKES recurring intermittently from roughly epoch 75 through epoch 320 (jumping back above 1.0 near epochs 200, 240, 282, and 303), caused by the fixed `lr=0.08` overshooting as the loss basin narrows -- only the FINAL ~30 epochs settle into a clean, spike-free descent. A follow-up check restricted to just the last 50 steps (epochs 301-350) still measured `noise_std=0.243`, only modestly lower than the full second-half number, because that window still

straddled two of the late spikes (~epoch 303 and ~320).

### Mathematics:

#### MATH

$\text{noise\_std} = \text{std}(\text{loss}[i+1] - \text{loss}[i])$  over steps  $i$  in the measurement window

epoch-matched (50 ep): FullBatch=0.031, Batch32=0.324, Batch8=0.328

step-matched (350 steps): FullBatch=0.264 (second half), 0.243 (last 50 steps only)

### Actual output:

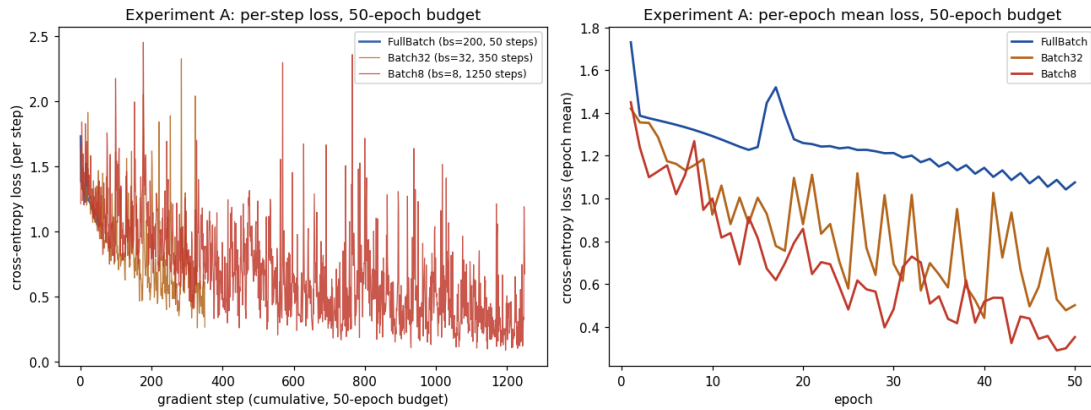
| config                   | steps | noise_std | train acc | test acc |
|--------------------------|-------|-----------|-----------|----------|
| FullBatch (50 ep)        | 50    | 0.031     | 0.600     | 0.438    |
| Batch32 (50 ep)          | 350   | 0.324     | 0.895     | 0.750    |
| Batch8 (50 ep)           | 1250  | 0.328     | 0.900     | 0.650    |
| FullBatch (step-matched) | 350   | 0.264     | 0.990     | 0.787    |

Actual output from this lesson's run.

### Line-by-line explanation

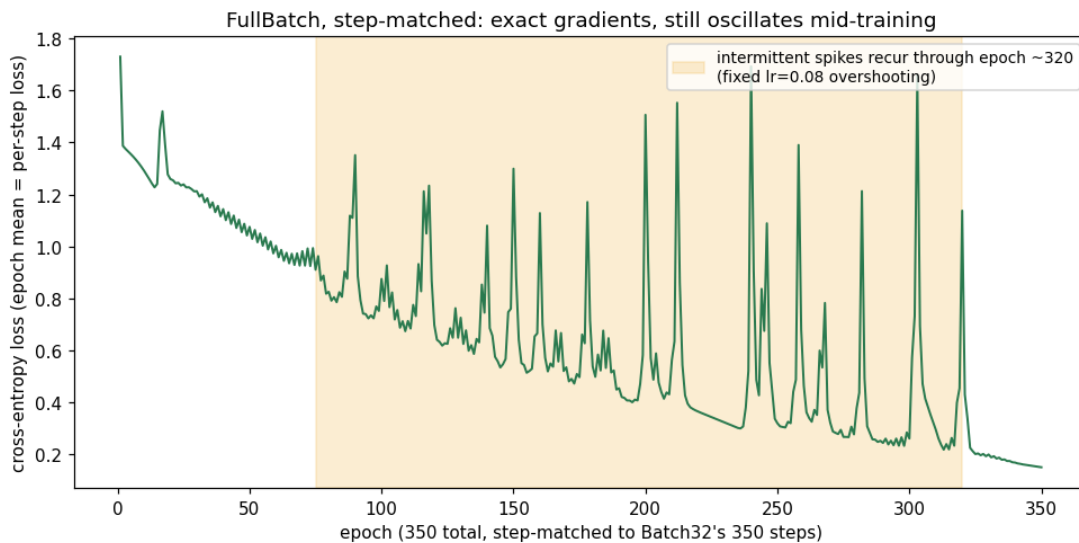
- Verified: Batch8 (1250 steps, the MOST of any config in Experiment A) still did NOT beat Batch32's test accuracy (0.650 vs. 0.750) despite having far more gradient updates -- its higher per-step noise (0.328 vs. Batch32's 0.324, and generally a smaller, noisier batch) plausibly cost it some of the benefit of those extra steps. More steps helps, but noisier steps are not free.
- The headline finding is precise, not the tempting-but-wrong shortcut: mini-batch training's real, practical advantage in this lesson was never 'noisy gradients generalize better' -- it is that mini-batches pack MANY MORE gradient updates into the same number of epochs, which matters enormously once a dataset is too large to complete even one full-batch epoch quickly.
- A fixed learning rate (today's  $\text{lr}=0.08$ , unchanged throughout both experiments) causes its own kind of late-training oscillation regardless of batch size -- a real, honest complication that this lesson did not set out looking for, and exactly what Day 46's learning-rate schedules (step decay, cosine annealing) are designed to fix.

### How it works, visually:



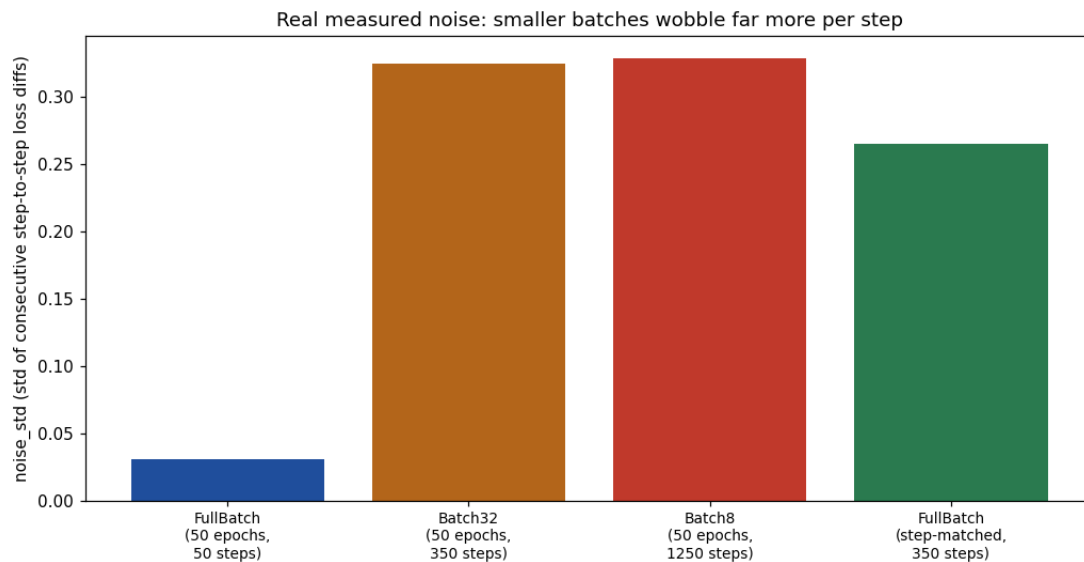
REAL GRAPH (script's own output) -- per-step loss (left) and per-epoch mean loss (right) for all three Experiment A configs, 50-epoch budget.

### How it works, visually:



REAL GRAPH (script's own output) -- FullBatch's full 350-epoch step-matched trace, showing genuine intermittent spikes (a fixed-lr effect) recurring until roughly epoch 320.

## How it works, visually:



REAL GRAPH (script's own output) -- measured noise\_std across all four runs; note step-matched FullBatch's noise is NOT near zero, despite exact gradients.

**Watch:** Mini Batch Gradient Descent (C2W2L01) — Andrew Ng's own explanation of mini-batch gradient descent from the Deep Learning Specialization -- the standard reference for the steps-per-epoch mechanics built from scratch in this lesson.

**Watch:** Gradient Descent Explained: Batch, Mini-Batch, and Stochastic (Simple) — A visual side-by-side comparison of batch, mini-batch, and stochastic gradient descent -- useful context for today's noise\_std measurements across all three regimes.

**Why choose this?** Running a step-matched follow-up experiment, instead of stopping at the first (misleading) epoch-matched result, is what turned a shallow 'mini-batch wins' headline into a precise, defensible finding about WHERE mini-batch's real advantage actually comes from.

**Why NOT this (tradeoffs)?** This lesson's own noise\_std metric could not, by itself, tell apart sampling noise from fixed-lr oscillation -- it took plotting the raw loss trace and checking a second window to untangle them. A metric that looks clean in a summary table is not automatically measuring just one thing.

## 6. What This Maps to in PyTorch

**Start here (no prior knowledge assumed):** Everything built by hand today -- shuffling, batch looping, the training loop itself -- collapses into a DataLoader object and an unchanged inner training loop in a real framework.

**Topic specification:** `torch.utils.data.DataLoader(dataset, batch_size=32, shuffle=True)` is PyTorch's built-in equivalent of today's `iterate_minibatches` -- it reshuffles indices every epoch when `shuffle=True`, and by default keeps a smaller final batch rather than dropping it (`drop_last=False`), matching this lesson's own default.

**Explanation:** The PyTorch inner loop 'for X\_batch, y\_batch in loader: ...' is EXACTLY the same code regardless of batch\_size -- unlike this lesson's from-scratch version, no new function or class is needed to change batch size, since DataLoader already handles both the full-batch and mini-batch cases identically.

**Real-world scenario:** This is why virtually every real PyTorch training script uses a DataLoader even for small datasets that WOULD fit in one full-batch pass -- the code is identical either way, so there is no cost to writing it the scalable way from the start.

### *Actual output:*

```
from torch.utils.data import DataLoader, TensorDataset

dataset = TensorDataset(X_tensor, y_tensor)
loader = DataLoader(dataset, batch_size=32, shuffle=True)

for epoch in range(epochs):
    for X_batch, y_batch in loader:          # reshuffles every epoch when
        optimizer.zero_grad()              # shuffle=True, exactly like
        out = model(X_batch)               # this lesson's
        loss = loss_fn(out, y_batch)       # iterate_minibatches(..., rng, ...)
        loss.backward()
        optimizer.step()
```

*Actual printed reference from this lesson's script run.*

### *What changes with a framework*

- `shuffle=True` — replaces today's `rng.permutation(m)` called fresh inside `iterate_minibatches` every epoch, with the reshuffling handled internally by `DataLoader`.
- `drop_last=False` (the default) — `DataLoader`'s equivalent of this lesson's own default behavior: keep a smaller final batch rather than silently discarding leftover examples.

**Why choose this?** Having built the shuffle-and-batch mechanics by hand, and personally measured the real steps-vs-noise tradeoff it creates, makes `DataLoader`'s two keyword arguments fully legible -- they are not magic, they are exactly the loop written out today.

**Why NOT this (tradeoffs)?** None of today's hand-written `iterate_minibatches/train_minibatch` code needs to be reused going forward -- its entire purpose was building this understanding once, exactly as every prior day's from-scratch code was.

## Interview Preparation — Technical Questions

Q: Why does mini-batch training reshuffle the dataset every epoch, rather than shuffling once at the very start of training?

A: If the data were shuffled only once, every epoch would see the exact same sequence of batches, which means the same examples would always end up grouped together. Any pattern in a fixed grouping (even an accidental one) could bias which directions the optimizer explores at a given point in EVERY epoch. Reshuffling every epoch (verified in this lesson's own printed output, where 3 epochs produced 3 different batch groupings from an identical toy dataset) ensures each epoch's batch composition is independent of every other epoch's.

Q: This lesson's Experiment A found full-batch training reaching only `train_acc=0.600` in 50 epochs, far below mini-batch's 0.895-0.900. Was this because mini-batch gradients are somehow better?

A: No -- Experiment B showed this directly. Full-batch takes exactly 1 gradient step per epoch, so a 50-epoch budget gives it only 50 total steps, while `batch_size=32` took 350 steps in the same 50 epochs. Given the SAME 350 total steps (350 epochs of full-batch training), full-batch reached `train_acc=0.990` and `test_acc=0.787` -- matching or beating the mini-batch result. The original gap was a STEPS problem, not evidence that full-batch gradients are worse.

Q: Before concluding the gap was about total steps, this lesson tried raising full-batch's learning rate. Why wasn't that the fix, and why was it still worth trying?

A: Trying `lr=0.2` (`train_acc=0.625`) and `lr=0.3` (`train_acc=0.350`, WORSE than the original `lr=0.08`) showed that no learning rate change came close to matching mini-batch's 0.895 `train_acc` under the same 50-epoch/50-step budget -- ruling out 'wrong learning rate' as the explanation. It was still worth trying because the alternative (assuming the real cause without checking) risks reporting a false explanation; a real, if small, negative result on the `lr` search is what made the steps-based explanation in Experiment B trustworthy rather than just convenient.

Q: The step-matched full-batch run measured `noise_std=0.264`, close to Batch32's 0.324 -- but full-batch gradients are exact. What does this measurement actually reflect?

A: It reflects real, visible loss OSCILLATION caused by a fixed learning rate (`lr=0.08`, unchanged for all 350 epochs) overshooting as the loss basin narrows during training -- not batch-sampling noise, which cannot exist in full-batch gradient descent since every step uses the entire dataset. Plotting the actual 350-epoch trace showed intermittent spikes recurring from roughly epoch 75 to epoch 320, confirming this is a deterministic optimization effect rather than randomness.

Q: Batch8 took more total gradient steps (1250) than any other config in Experiment A, yet its test accuracy (0.650) was worse than Batch32's (0.750). Why might more steps not translate into better generalization?

A: Batch8's per-step `noise_std` (0.328) was the highest of the three Experiment A configs, consistent with the smaller batch size producing noisier individual gradient estimates. More steps generally helps convergence, but each of Batch8's steps is a less reliable estimate of the true gradient direction than Batch32's -- the extra step count and the extra noise are both real effects pulling in opposite directions, and this lesson's honest result is that the noise cost outweighed the step-count benefit on this specific task.

Q: Why is `train_minibatch()` able to treat full-batch training as a special case, rather than needing separate code for it?

A: `iterate_minibatches(X, y_onehot, batch_size, rng)` with `batch_size` set to (or above) the full dataset size produces exactly one batch per epoch containing every example -- mechanically identical to what a plain 'for epoch: `net.forward(X)`; `net.backward(y_onehot)`' loop already did in every network from Day 37 through Day 44. No new logic is needed; full-batch is just the `batch_size=m` case of the same general loop.

## Interview Preparation — Theoretical Questions

Q: Given today's results, how would you correct someone who claims 'mini-batch training works better because the noise helps the model generalize'?

A: This lesson's own step-matched experiment complicates that claim directly: given an EQUAL number of total gradient steps, full-batch training (zero sampling noise) matched or beat mini-batch's test accuracy (0.787 vs. 0.750). The real, defensible claim is narrower and more mechanical: mini-batch's practical value is packing more gradient UPDATES into the same number of epochs (or the same wall-clock pass over data too large to process in one full-batch step), not that noisy gradients are inherently better for generalization on every task.

Q: This lesson measured 'noise\_std' as the standard deviation of consecutive step-to-step loss differences. What is the risk of treating a single summary metric like this as a complete measurement of 'how noisy is training'?

A: A single window of a metric can conflate genuinely different underlying mechanisms -- here, batch-sampling variance (present in mini-batch, absent in full-batch by construction) and fixed-learning-rate oscillation (which can appear in EITHER regime, including exact full-batch gradients). This lesson's step-matched full-batch run scored a surprisingly high noise\_std for a deterministic method, and only checking a second window (the final 50 steps) plus plotting the raw trace revealed the real cause. A metric that looks clean in a single table is not automatically measuring only what its name suggests.

Q: Why does an 'epoch' remain a meaningful unit for reporting training progress, even though this lesson found that comparing configs by epoch count alone was misleading?

A: An epoch (one full pass through the data) is still the natural unit for talking about how much of the DATASET a model has seen, and for comparing wall-clock cost when reading a full dataset is itself expensive (as with data too large to fit in memory). What this lesson showed is narrower: epoch count is NOT a fair proxy for how much LEARNING (gradient update) has happened when comparing across different batch sizes, since the number of steps per epoch varies by a factor of 25x between today's configs. Both units matter, for different questions.

Q: How does today's honest discovery of fixed-lr oscillation (in a fully deterministic full-batch run) connect to the motivation for Day 46's learning-rate schedules?

A: A fixed learning rate is tuned as one single number for the entire training run, but the loss landscape's local shape typically changes as training progresses -- often narrowing near a good minimum, as this lesson's step-matched run's intermittent spikes (recurring until roughly epoch 320)

suggest happened here. A learning-rate SCHEDULE (step decay, cosine annealing) reduces the step size over time specifically to avoid overshooting a shrinking basin late in training, without needing to guess one perfect fixed value in advance -- exactly the problem this lesson's own honest complication ran into.

## Key Takeaway and What's Next

**Key takeaway:** Mini-batch gradient descent, built from scratch as a reshuffle-then-loop generator wrapped in a generic training function, is not just 'full-batch training but noisier' -- an epoch-matched comparison made full-batch look dramatically worse (train\_acc 0.600 vs. 0.895), but a step-matched follow-up revealed the real cause: full-batch simply took far fewer total gradient steps (50 vs. 350) in the same epoch budget, and closed the gap entirely once given equal steps (train\_acc 0.990, test\_acc 0.787). Mini-batch's genuine practical advantage is packing many more updates into the same pass over the data, not 'noise helps' -- and Batch8's extra steps still didn't beat Batch32, since noisier steps are not free. An unplanned discovery -- a fixed  $\text{lr}=0.08$  causing real loss spikes late into a 350-epoch full-batch run, something that cannot be sampling noise since the gradient is exact -- previews tomorrow's topic directly.

### *Suggested watch order, if starting from zero*

- 1. "Mini Batch Gradient Descent (C2W2L01)" (Andrew Ng, Topic 5) -- the standard reference for the steps-per-epoch mechanics built from scratch in Topics 1-2.
- 2. "Gradient Descent Explained: Batch, Mini-Batch, and Stochastic (Simple)" (Topic 5) -- a visual side-by-side of all three regimes, useful before or after this lesson's own noise\_std measurements.

### *Day 46 preview*

- Learning-rate schedules -- step decay and cosine annealing, motivated directly by today's own honest discovery that a FIXED learning rate causes real loss oscillation late in a long training run, even with exact full-batch gradients.
- Revisiting WideTwoBlockConvNetRGB -- Day 44's wide two-block network never fully converged (train\_acc=0.995) under a fixed  $\text{lr}=0.08$ ; Day 46 retries it with a proper schedule instead of a single fixed value.